Testing the 21cmFast Output Volume

Initialisation

```
# Import python modules
import os
import sys
import numpy as np
from matplotlib import pyplot as plt, colors
from astropy import units as u, constants as c
from astropy.cosmology import FlatLambdaCDM as fmodel, z_at_value as getz
import h5py
import py21cmfast as p21c
import json

# Import local oskareor models
sys.path.append("..")
from oskareor.skalow_calc import SKAMath as omath, EoRCosmology as Cosmo, FileManager as ofm

# EoR colour map
EoR_colour = colors.LinearSegmentedColormap.from_list('mycmap',
    [(0, 'white'),(0.125, 'yellow'),(0.25, 'orange'),(0.325, 'red'),\
    (0.5, 'black'),(0.75, 'blue'),(1, 'cyan')])

eor_cmap = {
    "map": EoR_colour,
    "norm": colors.TwoSlopeNorm(0, -160, 40),
    "ticks": np.arange(-160, 41, 20),
    "bounds": np.linspace(-160, 40, 257),
    "dist": "uniform"
}

# Import the h5 File Data
path = ofmg.expand_path("~/oskareor.data/simulations/project1/fiducial_lightcone_simulation.
file = h5py.File(path)
data = np.array(file['lightcones']['brightness_temp'])
dist = np.array(file['lightcone_distances'])
inpd = p21c.io.h5.read_inputs(file)

print(list(file))
```

```
['InputParameters', 'global_quantities', 'lightcone_distances', 'lightcones', 'photon_noncor
```

```
print(json.dumps(inpd.asdict(), indent=4, sort_keys=True, default=str))
```

```
{
    "astro_options": {
        "CELL_RECOMB": false,
```

```
    "FIX_VCB_AVG": false,
    "HALO_SCALING_RELATIONS_MEDIAN": false,
    "HEAT_FILTER": "spherical-tophat",
    "HII_FILTER": "sharp-k",
    "INTEGRATION_METHOD_ATOMIC": "GAUSS-LEGENDRE",
    "INTEGRATION_METHOD_MINI": "GAUSS-LEGENDRE",
    "IONISE_ENTIRE_SPHERE": false,
    "LYA_MULTIPLE_SCATTERING": false,
    "M_MIN_in_Mass": true,
    "PHOTON_CONS_TYPE": "no-photoncons",
    "RECOMB_MODEL": "inhomogeneous",
    "USE_ADIABATIC_FLUCTUATIONS": true,
    "USE_CMB_HEATING": true,
    "USE_EXP_FILTER": false,
    "USE_LYA_HEATING": true,
    "USE_MINI_HALOS": false,
    "USE_TS_FLUCT": true,
    "USE_UPPER_STELLAR_TURNOVER": false,
    "USE_X_RAY_HEATING": true
},
"astro_params": {
    "ALPHA_ESC": -0.46,
    "ALPHA_STAR": 0.51,
    "ALPHA_STAR_MINI": 0.51,
    "ALPHA_UVB": 5.0,
    "A_LW": 2.0,
    "A_VCB": 1.0,
    "BETA_LW": 0.6,
    "BETA_VCB": 1.8,
    "CLUMPING_FACTOR": 2.0,
    "DELTA_R_HII_FACTOR": 1.1,
    "FIXED_VAVG": 25.86,
    "F_ESC10": -1.08,
    "F_ESC7_MINI": -2.0,
    "F_H2_SHIELD": 0.0,
    "F_STAR10": -1.42,
    "F_STAR7_MINI": -2.95,
    "HII_EFF_FACTOR": 30.0,
    "ION_Tvir_MIN": 4.69897,
    "L_X": 40.5,
    "L_X_MINI": 40.5,
    "MAX_DVDR": 0.2,
    "M_TURN": 8.46,
    "NU_X_BAND_MAX": 2000.0,
    "NU_X_MAX": 10000.0,
    "NU_X_THRESH": 500.0,
```

```
    "N_STEP_TS": 40,
    "PHOTONCONS_CALIBRATION_END": 3.5,
    "POP2_ION": 5000.0,
    "POP3_ION": 44021.0,
    "R_BUBBLE_MAX": 50.0,
    "R_BUBBLE_MIN": 0.620350491,
    "R_MAX_TS": 500.0,
    "SIGMA_LX": 0.5,
    "SIGMA_SFR_INDEX": -0.12,
    "SIGMA_SFR_LIM": 0.19,
    "SIGMA_STAR": 0.25,
    "T_RE": 20000.0,
    "UPPER_STELLAR_TURNOVER_INDEX": -0.6,
    "UPPER_STELLAR_TURNOVER_MASS": 11.447,
    "X_RAY_SPEC_INDEX": 1.0,
    "X_RAY_Tvir_MIN": 4.69897,
    "t_STAR": 0.34
},
"cosmo_params": {
    "OMb": 0.04897468161869667,
    "OMk": 0.0,
    "OMl": 0.6903585584544936,
    "OMm": 0.30964144154550644,
    "OMn": 0.0,
    "OMr": 8.6e-05,
    "OMtot": 1.0,
    "POWER_INDEX": 0.9665,
    "Y_He": 0.24,
    "hlittle": 0.6766,
    "wl": -1.0
},
"cosmo_tables": {
    "USE_SIGMA_8": "True",
    "ps_norm": 0.8102,
    "transfer_density": null,
    "transfer_vcb": null
},
"matter_options": {
    "DEXM_OPTIMIZE": false,
    "FILTER": "spherical-tophat",
    "HALO_FILTER": "spherical-tophat",
    "HMF": "ST",
    "KEEP_3D_VELOCITIES": false,
    "MINIMIZE_MEMORY": false,
    "PERTURB_ALGORITHM": "2LPT",
    "PERTURB_ON_HIGH_RES": false,
```

```
    "POWER_SPECTRUM": "EH",
    "SAMPLE_METHOD": "MASS-LIMITED",
    "SMOOTH_EVOLVED_DENSITY_FIELD": false,
    "SOURCE_MODEL": "E-INTEGRAL",
    "USE_FFTW_WISDOM": false,
    "USE_INTERPOLATION_TABLES": "hmf-interpolation",
    "USE_RELATIVE_VELOCITIES": false
},
"node_redshifts": [
    35.402256033675854,
    34.68848630752535,
    33.98871206620133,
    33.30265888843268,
    32.63005773375753,
    31.97064483701719,
    31.32416160491882,
    30.690354514626293,
    30.068975014339507,
    29.45977942582305,
    28.86252884884613,
    28.27698906749621,
    27.702930458329618,
    27.140127900323158,
    26.588360686591336,
    26.047412437834645,
    25.517071017484948,
    24.99712844851466,
    24.48738083187712,
    23.987628266546196,
    23.497674771123723,
    23.017328206984047,
    22.546400202925536,
    22.084706081299547,
    21.632064785587794,
    21.1882988093998,
    20.75323412686255,
    20.326700124375055,
    19.908529533701035,
    19.498558366373565,
    19.096625849385852,
    18.702574362142993,
    18.316249374649995,
    17.93749938691176,
    17.566175869521338,
    17.202133205413077,
    16.84522863275792,
```

16.495322188978353,
16.152276655861133,
15.815957505746212,
15.486232848770797,
15.162973381147843,
14.846052334458669,
14.535345425939873,
14.230730809744976,
13.932089029161743,
13.639302969766415,
13.352257813496488,
13.070840993624008,
12.794942150611774,
12.524453088835074,
12.259267734152035,
11.999282092305918,
11.744394208143058,
11.49450412563045,
11.249513848657305,
11.009327302605202,
10.773850296671768,
10.542990486933107,
10.316657340130499,
10.094762098167156,
9.877217743301134,
9.663938964020721,
9.454842121588943,
9.249845217244063,
9.0488678600432,
8.85183123533647,
8.658658073859284,
8.469272621430672,
8.283600609245758,
8.101569224750744,
7.923107083088965,
7.748144199106829,
7.576611959908657,
7.4084430979496645,
7.243571664656535,
7.081933004565231,
6.923463729965913,
6.768101696045014,
6.6157859765147204,
6.466456839720315,
6.320055725215996,
6.176525220799997,

```
        6.0358090399999975,
        5.897851999999998,
        5.762599999999999,
        5.629999999999999,
        5.5
    ],
    "random_seed": 20250303,
    "simulation_options": {
        "BOX_LEN": null,
        "CORR_LX": 0.2,
        "CORR_SFR": 0.2,
        "CORR_STAR": 0.5,
        "DELTA_R_FACTOR": 1.1,
        "DENSITY_SMOOTH_RADIUS": 0.2,
        "DEXM_OPTIMIZE_MINMASS": 100000000000.0,
        "DEXM_R_OVERLAP": 2.0,
        "DIM": null,
        "HALOMASS_CORRECTION": 0.89,
        "HII_DIM": 512,
        "INITIAL_REDSHIFT": 300.0,
        "MIN_LOGPROB": -12.0,
        "MIN_XE_FOR_FCOLL_IN_TAUX": 0.001,
        "NON_CUBIC_FACTOR": 1.0,
        "N_COND_INTERP": 200,
        "N_PROB_INTERP": 400,
        "N_THREADS": 8,
        "PARKINSON_G0": 1.0,
        "PARKINSON_y1": 0.0,
        "PARKINSON_y2": 0.0,
        "SAMPLER_BUFFER_FACTOR": 2.0,
        "SAMPLER_MIN_MASS": 100000000.0,
        "ZPRIME_STEP_FACTOR": 1.02,
        "Z_HEAT_MAX": 35.0
    }
}
```

```
print(data.shape)
```

```
(512, 512, 1024)
```

```
# Cosmology data
cosmo = inpd.cosmo_params.cosmo
btzs = getz(cosmo.comoving_distance, dist * u.Mpc).to_value()
print(btzs)
```

```
[ 5.50000001  5.5041941   5.50839227 ... 13.41934869 13.43317339
 13.44701797]
```

Global Parameters

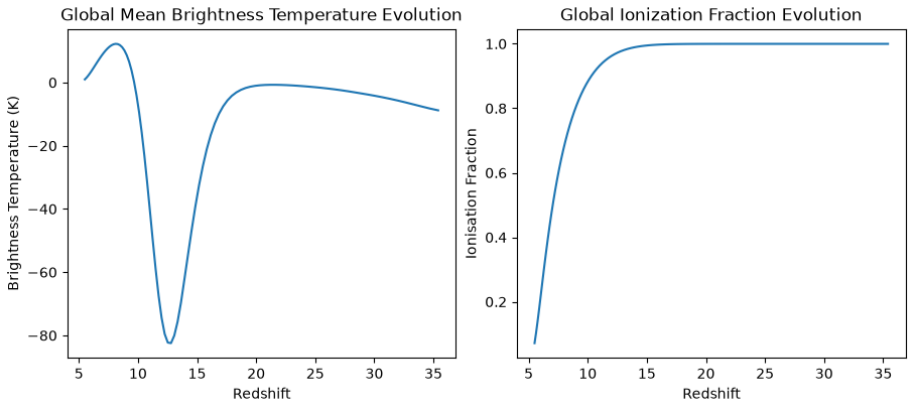
Printing global parameters against the node redshifts.

```
print(list(file['global_quantities']))
```

```
['brightness_temp', 'cumulative_recombinations', 'density', 'ionisation_rate_G12', 'kinetic_
```

```
nzs = np.array(inpd.node_redshifts)
gtbs = np.array(file['global_quantities']['brightness_temp'])
xh1s = np.array(file['global_quantities']['neutral_fraction'])
```

```
fig, ax = plt.subplots(nrows=1, ncols=2, layout="constrained")
fig.set_size_inches((9, 4))

ax[0].plot(nzs, gtbs)
ax[0].set_xlabel("Redshift")
ax[0].set_ylabel("Brightness Temperature (K)")
ax[0].set_title("Global Mean Brightness Temperature Evolution")

ax[1].plot(nzs, xh1s)
ax[1].set_xlabel("Redshift")
ax[1].set_ylabel("Ionisation Fraction")
ax[1].set_title("Global Ionization Fraction Evolution")

plt.show()
```

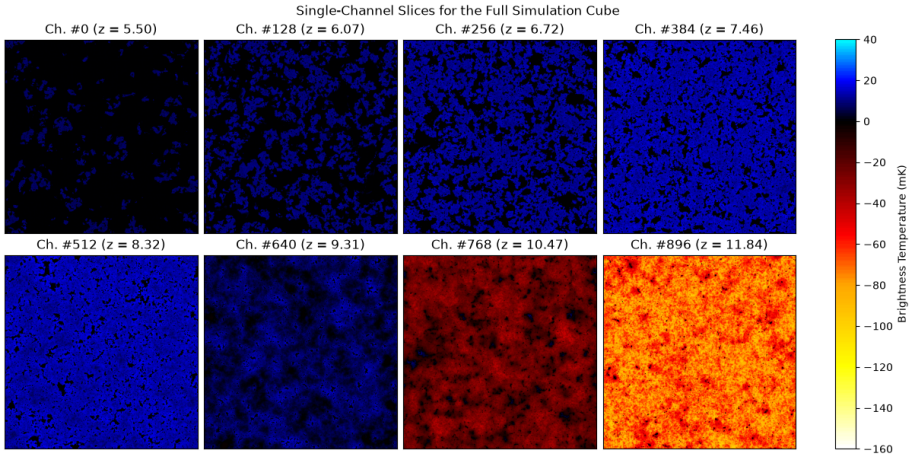


Concerningly it does seem like the ionisation fraction **does not** follow an expected logistic curve but instead a flat logarithmic curve. This could also be because of the nature of the simulation and the chosen redshifts.

Transverse Dimension Snapshots

Now we will see different snapshots of the saved lightcone volume at different redshifts.

```
channels = np.arange(8) * 128

fig, ax = plt.subplots(nrows=2, ncols=4, layout="constrained", squeeze=True)
fig.set_size_inches((12,6))
ax = ax.flatten()

fig.suptitle("Single-Channel Slices for the Full Simulation Cube")

for i, chn in enumerate(channels):
    im = ax[i].imshow(data[:,:,chn], cmap=eor_cmap["map"], norm=eor_cmap["norm"])
    ax[i].set_xticks([])
    ax[i].set_yticks([])
    ax[i].set_title(f"Ch. #{chn} (z = {btzs[chn]:3.2f})")


fig.colorbar(im, ax=ax.ravel().tolist(), label="Brightness Temperature (mK)", boundaries=eor

plt.show()
```



Plotting Lightcones

```
def custom_image_ticks(values, labels):
    scales = np.interp(labels, values, np.arange(values.shape[0]))
    return scales, labels

fig, ax = plt.subplots(layout="constrained")
fig.set_size_inches((12,6))

im = ax.imshow(data[:,256,:], cmap=eor_cmap["map"], norm=eor_cmap["norm"])
```

```
ax.set_xticks(*custom_image_ticks(btzs, np.arange(6, 14, 1)))
ax.set_yticks(*custom_image_ticks(np.arange(512)*2, np.arange(0, 1025, 128)))

nrows, ncols = im.get_size()
aspect_ratio = ncols / nrows
fig.colorbar(im, ax=ax, label="Brightness Temperature (mK)", boundaries=eor_cmap["bounds"],

plt.show()
```